M. Frisbie, *Building Browser Extensions*
https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_11

11. Extension and Browser APIs

Matt Frisbie[1]
(1) California, CA, USA

The WebExtensions API is the secret sauce of browser extensions. It allows them to reach into the web page and the browser to make changes, inspect and modify network traffic, and control aspects of the native browser's user interface.

Global API Namespace

All extensions are accessible from inside the global API namespace. This is either available via the `chrome.*` namespace, or the `browser.*` namespace.

- Chromium browsers and Safari favor the `chrome.*` namespace
- Firefox favors the `browser.*` namespace

All browsers support the `chrome.*` namespace, so prefer to use that in all extensions.

Promises vs. Callbacks

The WebExtensions API was written before `async/await` was an industry standard. The initial API implementation used callbacks to support asynchronous code execution. For example, the following snippet executes a callback after a message response is received:

```
chrome.runtime.sendMessage("msg", (response) => {
  console.log("Received a response!", response);
});
```

To align with modern coding conventions, most browsers have retrofitted their API methods to support both callbacks and promises. The above snippet can be refactored to use `async/await`:

```
const response = await chrome.runtime.sendMessage("msg");
console.log("Received a response!", response);
```

There are some things to know about making this change:

- Not all API methods have been adapted to support promises. Check your browser's documentation to see if it is supported. Callbacks will always work.
- For each method call, only use a callback or a promise, never both.
- Mozilla's `webextension-polyfill` (**https://github.com/mozilla/webextension-polyfill**) can be used to augment the WebExtensions API methods to be promise-based irrespective of the host browser's implementation.

Error Handling

Errors thrown from WebExtensions API methods must be handled differently based on your usage. If you use callbacks, when the method fails the `chrome.runtime.lastError` property will be defined only inside the callback handler:

```
chrome.tabs.executeScript(tabId, details, () => {
  if (chrome.runtime.lastError) {
    // Error handling here
  }
})
```

If you use `async/await`, the error can be caught with a `.catch()` or `try/catch` block:

```
chrome.tabs.executeScript(tabId, details).catch((e) => {
  // Error handling here
})
// or
try {
  await chrome.tabs.executeScript(tabId, details);
} catch(e) {
  // Error handling here
}
```

Context-restricted APIs

Not all APIs can be used everywhere:

- Some APIs such as `tabCapture` and `fileSystem` are restricted to the foreground, meaning they cannot be accessed from a background service worker.
- The Devtools API can only be used in a devtools context.

Events API

The "Events API" refers to the general pattern of how you can add event listeners to events fired in browser extensions. Most WebExtensions APIs utilize this format.

Format

Each event type has an interface to add, remove, and inspect listeners. For example, the `chrome.runtime.onMessage` interface has the following methods:

- `chrome.runtime.onMessage.addListener()`
- `chrome.runtime.onMessage.dispatch()`
- `chrome.runtime.onMessage.hasListener()`
- `chrome.runtime.onMessage.hasListeners()`
- `chrome.runtime.onMessage.removeListener()`

> **Warning** `dispatch()` can be used to forcibly dispatch an event, but it is not documented and therefore should not be relied upon.

These methods are shared by all event interfaces. `addListener()` is used to attach a handler function to run when an event is fired, and `hasListener()`, `hasListeners()`, and `removeListener()` are used to view and remove that same function.

The following example uses several of these methods to handle toolbar icon button clicks:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
```

```
  },
  "action": {},
  "permissions": []
}
```

Example 11-1a manifest.json

```
let count = 0;
function handler() {
  console.log("Handler executed!");
  if (++count > 4) {
    chrome.action.onClicked.removeListener(handler);
  }
}
// false
console.log(chrome.action.onClicked.hasListener(handler));
chrome.action.onClicked.addListener(handler);
// true
console.log(chrome.action.onClicked.hasListener(handler));
```

Example 11-1b background.js

This example sets a listener for `onClicked` events, listens for five events, and then de-registers itself.

Event Filtering

If you wished to restrict an event handler to a certain domain, you would need to perform filtering inside the handler:

```
chrome.webNavigation.onCommitted.addListener((event) => {
  if (!event.url.match(/^https:\/\/*.wikipedia\.org/)) {
    return;
  }
  // Handle filtered event
});
```

Instead, you can instruct the browser to declaratively filter events that match certain URLs. The browser natively performing this filtering will give a considerable performance boost. The above code can be refactored as follows:

```
chrome.webNavigation.onCommitted.addListener((event) => {
  // Handle filtered event
}, {
  url: [
    { hostSuffix: "wikipedia.org" }
```

```
    ]
  });
```

WebExtensions API Quick Reference

This section is a comprehensive list of extension APIs. Each API includes a short snippet demonstrating common usage and links to the full documentation.

> **Tip** The companion extension to this book, *Browser Extension Explorer*, has interactive demos and source code for most of these APIs. Get it here: **https://buildbrowserextensions.com/b2x**

Permissions

- Permissions API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/permissions/**
- Permissions API – MDN: **https://developer.mozilla.org/en-US/docs/Web/API/Permissions_API**

The `chrome.permissions` API allows you to inspect current permissions and add or remove permissions. You can also add handlers that fire when permissions are added or removed.

```
chrome.action.onClicked.addListener(() => {
  // Can only request permissions after user action
  const permissionsGranted =
    await chrome.permissions.request({
      permissions: ['tabs']
    });
});
const hasPermissions = await chrome.permissions.contains({
  permissions: ['tabs']
};
```

Messaging

- Chrome Messaging Overview – Chrome Developers: **https://developer.chrome.com/docs/extensions/mv3/messaging/**
- `runtime` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/runtime/**

- `runtime` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/runtime/**
- `tabs` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/tabs/**
- `tabs` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/tabs/**

The extension messaging API is used to send messages between parts of the extension, or between the extension and an outside entity (such as another extension, the active web page, or an outside program).

One-off Messages

The `chrome.runtime.sendMessage()` method can be used to send single messages between extension components. It can only be used to send messages *from* a content script. Listeners can send a message back inside the message handler.

```
// Sender
chrome.runtime.sendMessage(msg, (response) => {
  // Handle response
});
// Receiver
chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
  // Handle initial message from sender
  sendResponse(responseMsg);
});
```

To send a one-off message to a specific content script, you must instead use `chrome.tabs.sendMessage()` and specify the `tabId`:

```
// Sender
chrome.tabs.sendMessage(tabId, msg, (response) => {
  // Handle response
});
// Receiver (content script)
chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
  // Handle initial message from sender
  sendResponse(responseMsg);
});
```

Opening Message Ports

Persistent message ports can be opened to connect two extension components that will be sending a large volume of message back and forth. It can only be used to connect a port *from* a content script.

```
// Endpoint 1
const port1 = chrome.runtime.connect({name: "portName"});
// Outgoing
port1.postMessage(msg);
// Incoming
port1.onMessage.addListener((msg) => {});
// Endpoint 2
let port2 = null;
chrome.runtime.onConnect.addListener((p) => {
  port2 = p;
});
// Outgoing
port2.postMessage(msg);
// Incoming
port2.onMessage.addListener((msg) => {});
```

To connect a port to specific content script, you must instead use `chrome.tabs.connect()` and specify the `tabId`:

```
// Endpoint 1
const port1 = chrome.tabs.connect(tabId, {name: "portName"});
// Outgoing
port1.postMessage(msg);
// Incoming
port1.onMessage.addListener((msg) => {});
// Endpoint 2 (content script)
let port2 = null;
chrome.runtime.onConnect.addListener((p) => {
  port2 = p;
});
// Outgoing
port2.postMessage(msg);
// Incoming
port2.onMessage.addListener((msg) => {});
```

Messaging Native Applications

Extensions can exchange messages with native applications. If the name of the native application is known (e.g., `com.my_company.my_application`), the extension can target it in a similar way to content scripts:

```
chrome.runtime.sendNativeMessage(
  applicationName, msg, (response) => {});
chrome.tabs.connectNative(
  applicationName, {name: "portName"});
```

An example of this is shown in the *Cross-Browser Extensions* chapter. Safari extensions use native messaging to connect the extension to the native iOS or macOS app.

Messaging Other Extensions

To send a one-off message to another extension, you can pass the extension's ID to `chrome.runtime.sendMessage()`:

```
// Sender
chrome.runtime.sendMessage(extensionId, msg, (response) => {
  // Handle response
});
// Receiver (foreign extension)
chrome.runtime.onMessageExternal.addListener((msg, sender, sendResponse) => {
  // Handle initial message from sender
  sendResponse(responseMsg);
});
```

To connect a port to another extension, you can use `chrome.runtime.connect()` and specify the `extensionId`:

```
// Extension 1
const port1 = chrome.runtime.connect(extensionId, {name: "portName"});
// Outgoing
port1.postMessage(msg);
// Incoming
port1.onMessage.addListener((msg) => {});
// Extension2
let port2 = null;
chrome.runtime.onConnectExternal.addListener((p) => {
  port2 = p;
});
// Outgoing
port2.postMessage(msg);
// Incoming
port2.onMessage.addListener((msg) => {});
```

This method also allows for regular web pages (not content scripts) to send messages to your extension in the exact same way. The API is exposed in the web page's JavaScript runtime, and your extension can receive messages if the following conditions are true:

- The host page knows the ID of your extension and passes it to `runtime.connect()` or `runtime.sendMessage()`
- Your extension whitelists the web page URL under the manifest's `externally_connectable` field

Storage

- `storage` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/storage/**
- `storage` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/storage**

The extension storage API is a simple but powerful key/value storage. There are four main storage options you can use:

- `storage.local` stores values only on the local browser
- `storage.sync` stores values that are shared between the authenticated browser session
- `storage.session` stores values in memory that will be discarded when the browser closes
- `storage.managed` is a read-only store intended for enterprise users

`local`, `sync`, and `session` all share the following:

- With only the `storage` permission, default maximum storage size for each is 5 MB. This limit can be removed with the additional `unlimitedStorage` permission.
- Each features only a `get()` and `set()` method. These methods are asynchronous, and both can read and write in parallel.
- You can listen for changes to each by setting an `onChanged` handler.
- Plain JavaScript objects can be stored and retrieved as-is, there is no need to serialize/deserialize.

The following example shows a parallelized `get/set` and `onChanged` handler:

```
chrome.storage.onChanged.addListener(console.log);
```

```
await chrome.storage.local.set({ foo: "tmp" });
// { foo: { newValue "tmp" } }
await chrome.storage.local.set({ foo: "bar", baz: "qux" });
// {
//   foo: { oldValue: "tmp", newValue "bar" }
//   baz: { newValue "qux" }
// }
console.log(await chrome.storage.local.get(["foo", "baz"]));
// { foo: "bar", baz: "qux" }
```

Authentication

- `identity` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/identity/**
- `identity` API – `MDN`: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/identity**

The `chrome.identity` API is used to manage authentication state and OAuth flows. Google Chrome supports extra methods that allow for native authentication inside the browser. All browsers support the use of `launchWebAuthFlow` for generic OAuth flows.

```
// Native browser OAuth
chrome.identity.getAuthToken(
  { interactive: true },
  (token) => {
    if (token) {
      const info = await chrome.identity.getProfileUserInfo(
        { accountStatus: "ANY" }
      );
    }
  }
);
// Supports generic OAuth
chrome.identity.launchWebAuthFlow(
  {
    url: authUrl,
    interactive: true,
  },
  (redirectUrl) => {
    if (redirectUrl) {
    }
  }
}
```

Note This API is covered in depth in the *Networking* chapter.

Network Requests

- `declarativeNetRequest` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/declarativeNetRequest/**
- `webRequest` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/webRequest/**
- `webRequest` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/webRequest**
- `webNavigation` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/webNavigation/**
- `webNavigation` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/webNavigation**

The network request APIs include `declarativeNetRequest`, `webRequest`, and `webNavigation`:

- `declarativeNetRequest` is used to declaratively instruct the browser to manage page traffic according to a set of rules defined by the extension.
- `webRequest` is used to imperatively control page traffic with JavaScript handlers
- `webNavigation` is used to inspect tab-level navigation events

```
// Using DNR to block traffic to wikipedia.org
chrome.declarativeNetRequest.updateDynamicRules({
  addRules: [{
    id: 1,
    priority: 1,
    action: {
      type: "block"
    },
    condition: {
      urlFilter: "https://www.wikpedia.org",
    },
  }]
});
// Using webRequest to block traffic to wikipedia.org
chrome.webRequest.onBeforeRequest.addListener(
  (details) => {
    return {
```

```
      cancel: true
    };
  },
  { urls: ["*://www.wikipedia.org/*"] },
  ["blocking"]
);
// Log all tab traffic with webNavigation
chrome.webNavigation.onCompleted.addListener((details) => {
  console.log(details.url);
});
```

Note These APIs are covered in depth in the *Networking* chapter.

Internationalization

- `i18n` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/i18n/**
- `i18n` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/i18n**

The `chrome.i18n` API allows you to determine system languages as well as read messages out of your `_locales/_code_/messages.json` files. The following example reads the active system language and retrieves a message from that language:

```
chrome.i18n.getUILanguage();
chrome.i18n.getMessage("foo_message");
```

Browser and System Control

The WebExtensions API offers a number of methods for inspecting and controlling the host system or UI.

Power

- `power` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/power/**

The `chrome.power` API allows the extension to prevent the host system from going to sleep.

```
// Keeps the system active,
// but allows the screen to be dimmed or turned off.
chrome.power.requestKeepAwake("system");
```

```
// Keeps the screen and system active
chrome.power.requestKeepAwake("display");
// Allows the host to behave normally
chrome.power.releaseKeepAwake();
```

Omnibox

- `omnibox` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/omnibox/**
- `omnibox` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/omnibox**

The `chrome.omnibox` API allows you to embed a special search interface into the browser's URL bar (Figure **11-1**). You can control the keywork that triggers the interface, which results appear, how the results are presented, and what happens when a result is selected.

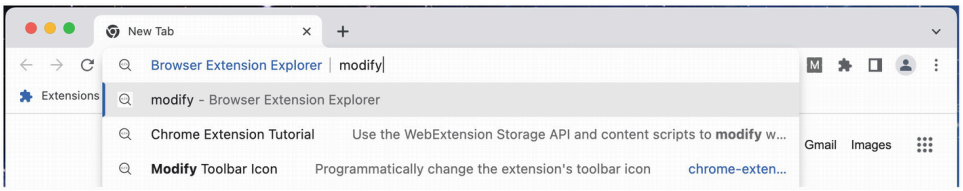

Figure 11-1 Example omnibox results

The following example populates the omnibar with a suggestion list, filtered by text match. If one is selected, it navigates to the URL.

```
const suggestions = [
  {
    content: url1,
    description: `
      ${title1}
      <dim>${subtitle1}</dim>
      <url>${url1}</url>`,
  },
  {
    content: url2,
    description: `
      ${title2}
      <dim>${subtitle2}</dim>
      <url>${url2}</url>`,
  }
];
chrome.omnibox.onInputChanged.addListener(
```

```
  (text, suggest) => {
    suggest(suggestions.filter(s => s.content.includes(text))
  }
);
chrome.omnibox.onInputEntered.addListener(
  (text, disposition) => {
    // Navigate to selected url
    chrome.tabs.update(activeTabId, { url: text });
  });
```

Action

- `action` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/action/**
- `action` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/action**

The `chrome.action` API allows you to control the extension toolbar icon, including appearance, title text, badge content, badge color, popup page, and click handlers (Figure 11-2).

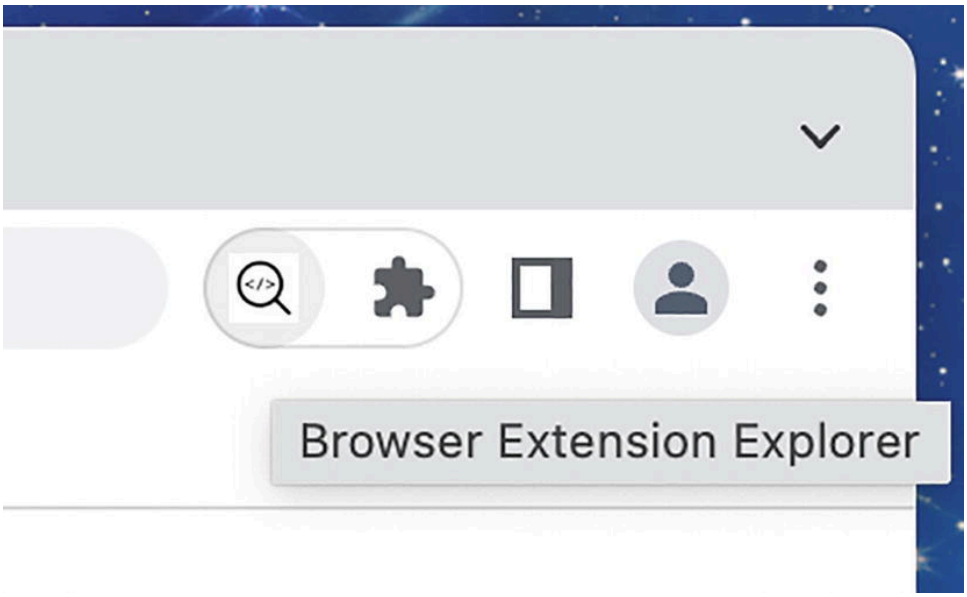

Figure 11-2 The toolbar icon button showing the title text

```
chrome.action.onClicked.addListener(() => {})
chrome.action.setTitle({ title: "foo"});
chrome.action.setBadgeText({ title: "bar"});
chrome.action.setPopup({
  popup: chrome.runtime.getURL("new_popup.html")
});
```

Notifications

- `notifications` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/notifications/**
- `notifications` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/notifications**

The `chrome.notification` API allows an extension to display rich notifications to the user using the host operating system's notification mechanism (Figure **11-3**).

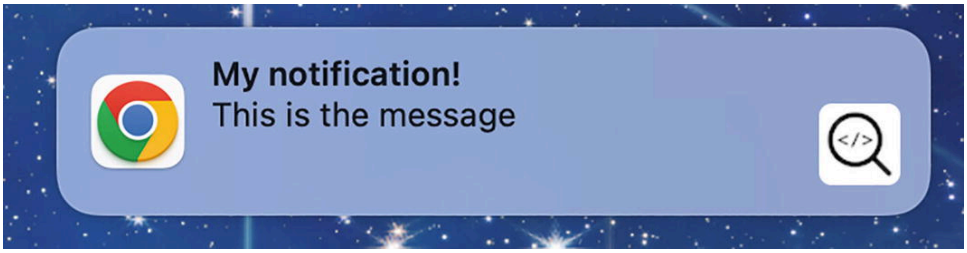

Figure 11-3 An example extension notification

```
chrome.notifications.create(
  "NOTIFICATION_ID",
  {
    type: "basic",
    iconUrl: "/img.png",
    title: "My notification!",
    message: "This is the message",
    priority: 2
  }
);
```

Context Menu

- `contextMenus` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/contextMenus/**
- `menus` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/menus**

The `chrome.contextMenus` API allows an extension to add interactive options to right click and context menus throughout the browser (Figure **11-4**).

Figure 11-4 An expanded extension context menu

```
chrome.contextMenus.create({
  id: "demo-radio",
  title: "Select a radio button...",
  contexts: ["all"],
});
for (let i of [1, 2, 3, 4, 5]) {
  chrome.contextMenus.create({
    id: `demo-radio-${i}`,
    parentId: "demo-radio",
    title: `Radio ${i}`,
    contexts: ["all"],
    type: "radio",
  });
}
chrome.contextMenus.create({
  id: "demo-media",
  title: "You right clicked a media element",
  contexts: ["image", "video", "audio"],
});
chrome.contextMenus.onClicked.addListener((info, tab) => {});
```

Page and Screen Capture

- `pageCapture` API – Chrome Developers: https://developer.chrome.com/docs/extensions/reference/pageCapture/
- `tabCapture` API – Chrome Developers: https://developer.chrome.com/docs/extensions/reference/tabCapture/

- `tabCapture` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/tabs/captureTab**
- `desktopCapture` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/desktopCapture/**
- `captureVisibleTab` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/tabs/#method-captureVisibleTab**
- `captureVisibleTab` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/tabs/captureVisibleTab**

Extensions are able to capture the system's current state in a handful of different ways: by saving the HTML, taking images of the active tab, or capturing media from the current desktop.

```
// Save a specific tab as MHTML
chrome.pageCapture.saveAsMHTML(
  {
    tabId: 123
  }
)
// Captures the visible area of the currently active tab.
chrome.action.onClicked.addListener(() => {
  chrome.tabCapture.capture(
    { video: true },
    (stream) => {}
  );
});
// Captures content of screen, individual windows or tabs.
chrome.desktopCapture.chooseDesktopMedia(
  sources: ["screen"],
  (streamId, options) => {}
);
// Captures the visible area of the currently active tab
// in the specified window.
chrome.tabs.captureVisibleTab(
  (dataUrl) =>{}
);
```

Proxy

- `proxy` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/proxy/**

- `proxy` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/proxy**

Use the `chrome.proxy` API to manage the browser's proxy settings.

> **Note** Per MDN: *Google Chrome provides an extension API also called "proxy" which is functionally similar to this API, in that extensions can use it to implement a proxying policy. However, the design of the Chrome API is completely different to this API. Because this API is incompatible with the Chrome proxy API, this API is only available through the browser namespace.*

```
chrome.proxy.settings.set(
  {
    value: {
      mode: "fixed_servers",
      rules: {
        proxyForHttp: {
          scheme: "socks5",
          host: "192.168.1.2",
        },
        bypassList: ["wikipedia.org"],
      },
    },
    scope: "regular",
  },
  () => {}
);
```

Browser State Management

The WebExtensions API gives you a broad set of tools for managing the state of the user's browser.

Font Settings

- `fontSettings` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/fontSettings/**

The API allows an extension to manage for some font settings to depend on certain generic font families and language scripts.

```
chrome.fontSettings.getFont(
  { genericFamily: 'standard', script: 'Egyp' },
```

```
    (details) => {}
  );
  chrome.fontSettings.setFont(
    { genericFamily: 'serif', script: 'Jpan',
      fontId: 'Comic Sans' }
  );
```

Content Settings

- `contentSettings` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/contentSettings/**

This API allows you to customize the browser's behavior on a per-site basis instead of globally: enabling or disabling cookies, JavaScript, and plugins. Each of the following properties has a `get()`, `set()`, and `clear()` method:

- `automaticDownloads`
- `camera`
- `cookies`
- `fullscreen`
- `images`
- `javascript`
- `location`
- `microphone`
- `mouselock`
- `notifications`
- `plugins`
- `popups`
- `unsandboxedPlugins`

Cookies

- `cookies` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/cookies/**
- `cookies` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/cookies**

The `chrome.cookies` API is a create/read/update/destroy interface for the browser's cookies.

```
const cookies = await chrome.cookies.getAll();
```

```
chrome.cookies.set({
    "name": "foo",
    "url": "https://www.wikipedia.org",
    "value": "bar"
});
chrome.cookies.remove(
    "name": "foo",
    "url": "https://www.wikipedia.org"
});
chrome.cookies.onChanged.addListener(() => {});
```

Bookmarks

- `bookmarks` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/bookmarks/**
- `bookmarks` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/bookmarks**

The `chrome.bookmarks` API is a create/read/update/destroy interface for the browser's bookmarks. It includes methods to manage the hierarchical nature of bookmarks.

```
chrome.bookmarks.create(
  {'title': 'Wikipedia', 'url': 'https://www.wikipedia.org'}
);
const allBookmarks = await chrome.bookmarks.getTree();
const matchingNodes =
  await chrome.bookmarks.search({ query: 'wikipedia' });
```

History

- `history` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/history/**
- `history` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/history**

The `chrome.history` API is a create/read/update/destroy interface for the browser's browsing history.

```
chrome.history.deleteAll();
const allPages = await chrome.history.search();
// Wikipedia pages in last 30 minutes
const pages = await chrome.history.search({
```

```
    text: 'wikipedia',
    startTime: Date.now() - (30 * 60 * 1E3)
  });
```

Downloads

- `downloads` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/downloads/**
- `downloads` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/downloads**

The `chrome.downloads` API is a create/read/update/destroy interface for the browser's downloads. "Creating" a download means forcing the browser to download from a specified URL.

```
// Start a download
chrome.downloads.download({
  url: "https://www.wikipedia.org/portal/wikipedia.org/assets/img/Wikipedia-logo-v2@2x.
});
const allDownloads = await chrome.downloads.search({});
```

Top Sites

- `topSites` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/topSites/**
- `topSites` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/topSites**

This API allows an extension to read the "top sites" (most visited sites) displayed on the new tab page.

```
const topSites = await chrome.topSites.get();
```

Browsing Data

- `browsingData` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/browsingData/**
- `browsingData` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/browsingData**

This API allows an extension to programmatically remove browsing data, which encompasses a considerable number of categories.

```
chrome.browsingData.remove({
  // Last 24 hours
  "since": (Date.now() - 24 * 60 * 60 * 1E3)
}, {
  "appcache": true,
  "cache": true,
  "cacheStorage": true,
  "cookies": true,
  "downloads": true,
  "fileSystems": true,
  "formData": true,
  "history": true,
  "indexedDB": true,
  "localStorage": true,
  "passwords": true,
  "serviceWorkers": true,
  "webSQL": true
});
```

Sessions

- `sessions` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/sessions/**
- `sessions` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/sessions**

This API allows an extension to programmatically inspect and reopen recently closed tabs.

```
const recentlyClosed =
  await chrome.sessions.getRecentlyClosed();
// Restores the most recently closed tab
chrome.sessions.restore();
```

Tabs and Windows

- `tabs` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/tabs/**
- `tabs` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/tabs**

- `windows` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/windows/**
- `windows` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/windows**
- `tabGroups` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/tabGroups/**

These APIs allow a browser to totally control the tabs and windows that appear inside a browser. It can perform tasks such as create, remove, inspect, modify, rearrange, pin, and mute.

```
const [activeTab] = await chrome.tabs.query({
  active: true,
  currentWindow: true
});
chrome.tabs.create({
  active: false,
  url: "https://www.wikipedia.org",
});
// Close a tab
chrome.tabs.remove(tabId);
// Reload a tab
chrome.tabs.reload(tabId);
// Create a new window
chrome.windows.create({
  focused: true,
  url: "https://www.wikipedia.org ",
});
// Close a window
chrome.windows.remove(tabId);
const allTabGroups = await chrome.tabGroups.query();
// Update a tab group
chrome.tabGroups.update(
  groupId,
  { title: "foo" }
);
```

Debugger

- `debugger` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/debugger/**

The `chrome.debugger` API allows you to programmatically control the browser's debugger. With it, you can do almost anything that the Devtools API is

capable of, including querying the response body, if the browser's developer tools interface is not open.

```
chrome.debugger.attach(
  tabId,
  "1.0",
  () => {}
);
chrome.debugger.sendCommand(
  tabId,
  "Debugger.enable"
);
chrome.debugger.sendCommand(
  tabId,
  "Debugger.setBreakpointByUrl",
  {
    lineNumber: 10,
    url: 'https://www.wikipedia.org/script.js'
  });
```

Search

- `search` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/search/**

This API allows you to programmatically execute searches in the browser's default search provider.

```
chrome.search.query(
  { disposition: "NEW_TAB", text: "wikipedia" });
```

Alarms

- `alarms` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/alarms/**
- `alarms` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/alarms**

This API is used to schedule code to run in the future. It is similar in concept to `setTimeout()` and `setInterval()`, but it is able to wake a service worker.

```
// Fire alarm event in 1 minute
```

```
chrome.alarms.create({ delayInMinutes: 1 });
// Fire alarm event every 5 minutes
chrome.alarms.create({ periodInMinutes: 5 });
// Fire alarm event in 90 seconds
chrome.alarms.create({ when: (Date.now() + 90 * 1E3) });
chrome.alarms.onAlarm.addListener((alarm) => {})
```

Scripting

- `scripting` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/scripting/**
- `scripting` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/scripting**

  This API is used to programmatically inject JavaScript or CSS into the page.

```
function fooScript() {}
// Inject a function
chrome.scripting.executeScript({
  target: { tabId: 123 },
  function: fooScript
});
// Inject a script file
chrome.scripting.executeScript({
  target: { tabId: 123 },
  files: ['script.js']
});
// Inject CSS string
chrome.scripting.insertCSS({
  target: { tabId: 123 },
  css: 'body { background-color: red; }'
});
// Inject CSS file
chrome.scripting.insertCSS({
  target: { tabId: 123 },
  files: ['styles.css']
});
```

DOM

- `dom` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/dom/**

The `dom` API has a single method, `openOrClosedShadowRoot()`, used to get the open shadow root or the closed shadow root hosted by the specified element.

```
chrome.dom.openOrClosedShadowRoot(el);
```

Text to Speech

- `tts` API – Chrome Developers: https://developer.chrome.com/docs/extensions/reference/tts

The `tts` API allows you to instruct your browser to speak text aloud using it's native text-to-speech engine.

```
// Browser will speak the text aloud
chrome.tts.speak('Foo', {'lang': 'en-US', 'rate': 2.0});
// Waits for previous speech to finish
chrome.tts.speak('Bar', {'enqueue': true});
// Stops the speech immediately
chrome.tts.stop();
```

Privacy

- `privacy` API – Chrome Developers: https://developer.chrome.com/docs/extensions/reference/privacy/

This API allows an extension to get and set a user's privacy controls.

```
chrome.privacy.services.searchSuggestEnabled.get(
  {},
  (details) => {
    if (details.levelOfControl ===
        'controllable_by_this_extension') {
      chrome.privacy.services.searchSuggestEnabled.set({
        value: true }
      );
    }
  });
```

Idle

- `idle` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/idle/**

  This API indicates when the host system has gone idle.

```
// Check system state every 30 seconds
chrome.idle.queryState(30);
chrome.idle.onStateChanged.addListener(() => {})
```

Devtools

- Devtools API – Chrome Developers: **https://developer.chrome.com/docs/extensions/mv3/devtools/**
- Devtools API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Extending_the_developer_tools**

The Devtools API allows you to add custom interfaces to the browser's developer tools interface. These interfaces are granted special access to APIs only accessible from inside the developer tools.

```
// Create a devtools panel
chrome.devtools.panels.create(
  "Panel title",
  "path/to/icon.png",
  "panel.html"
);
// Sniff active page traffic
chrome.devtools.network.onNavigated.addListener((url) => {
});
```

Note This topic is covered in the *Devtools Pages* chapter.

Extension Introspection

- `extension` API – Chrome Developers: **https://developer.chrome.com/docs/extensions/reference/extension/**
- `extension` API – MDN: **https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/extension**

This API is a collection of utilities. Many of the methods can be used to inspect the internal configuration of the running extension.

```
const popupUrl = chrome.extension.getURL("popup.html");
const allViews = chrome.extension.getViews();
const popups = chrome.extension.getViews({ type: "popup" });
```

Extension Management

- `management` API – Chrome Developers:
  https://developer.chrome.com/docs/extensions/reference/management/
- `runtime` API – Chrome Developers:
  https://developer.chrome.com/docs/extensions/reference/runtime/

Extensions are able to control the fundamentals of own operation, including reloading the extension, checking for updates, self-uninstalling, and setting the uninstall URL.

```
const selfInfo = chrome.management.getSelf();
// Uninstalls the running extension
chrome.management.uninstallSelf();
const manifest = chrome.runtime.getManifest();
// Reloads the extension
chrome.runtime.reload();
// Specifies the URL to direct the user to
// after uninstall
chrome.runtime.setUninstallURL(
  "https://www.wikipedia.org");
```

System State

- `system` APIs – Chrome Developers:
  - https://developer.chrome.com/docs/extensions/reference/system_cpu/
  - https://developer.chrome.com/docs/extensions/reference/system_display/
  - https://developer.chrome.com/docs/extensions/reference/system_memory/
  - https://developer.chrome.com/docs/extensions/reference/system_storage/

The system API allows you to inspect certain details about the host operating system. The `cpu`, `display`, `memory`, and `storage` properties are all getters that return a JavaScript object filled with whatever the host system provides.

Enterprise Only

The following APIs are only usable by extensions force-installed by enterprise policy and are not covered in this chapter:

- `enterprise.deviceAttributes`
- `enterprise.hardwarePlatform`
- `enterprise.networkingAttributes`
- `enterprise.platformKeys`

Firefox Only

The following APIs are only usable on Firefox and are not covered in this chapter:

- `captivePortal`
- `contextualIdentites`
- `dns`
- `find`
- `menus`
- `pcks11`
- `sidebarAction`
- `theme`
- `userScripts`

Chrome OS Only

The following APIs are only usable on ChromeOS and are not covered in this chapter:

- `accesibilityFeatures`
- `audio`
- `certificateProvider`
- `fileBrowserHandler`
- `fileSystemProvider`
- `input.ime`
- `loginState`
- `platformKeys`
- `printing`
- `printingMetrics`
- `restart`
- `restartAfterDelay`
- `vpnProvider`

Deprecated

The following APIs are deprecated and should not be used:

- `instanceID`
- `gcm`

Summary

In this chapter, we covered all the various idiosyncrasies of the WebExtensions API, including promises and callbacks, error handling, and the events API. Finally, we went through each of the APIs, learning a bit about what they are for and seeing a code snippet for each.

In the next chapter, we will discuss the permissions that are required to enable these APIs as well as the best ways to manage permissions in an extension.